

CS 499 Milestone Two: Enhancement One

Zoe Render

08/04/2026

Software Design and Engineering Artifact Narrative

1. Artifact Description

The artifact selected for this enhancement is a Raspberry Pi thermostat controller written in Python. The original program was created during earlier computer science coursework before CS 499. It uses GPIO buttons to change the thermostat mode and target temperature, PWM LEDs to represent heating and cooling behavior, an AHTx0 temperature sensor to obtain environmental data, an I2C character LCD to display the current mode and temperatures, and UART serial communication to transmit timestamped thermostat data. The original artifact already demonstrated a complete embedded-system workflow, but its structure relied heavily on global variables and standalone functions, leaving several opportunities for stronger software engineering practices.

2. Justification for Inclusion and Description of Improvements

I selected this artifact for my ePortfolio because it demonstrates software interacting directly with hardware while also requiring state management, event handling, sensor processing, user input, display output, and serial communication. These features make it a strong artifact for showing that software engineering is more than writing code that produces output. The program must remain understandable, reliable, maintainable, and safe while coordinating multiple hardware components.

The enhancement focused on improving the internal design without changing the project into an entirely different application. I reorganized the thermostat into an object-oriented `ThermostatController` class, which encapsulates thermostat state and hardware behavior instead of relying on global state. I also added a `ThermostatMode` enumeration so mode values are explicit and less susceptible to invalid string values. Hardware settings, temperature limits, hysteresis, serial settings, and timing values were moved into named constants, making the program easier to understand and modify.

The enhanced artifact also improves defensive programming. The set temperature is now constrained to a defined range of 55 to 85 degrees Fahrenheit, preventing unlimited input changes. Sensor readings are validated before they are used by the thermostat logic, and clearly invalid readings trigger controlled error handling. UART transmission errors are isolated so a serial communication problem does not automatically terminate the entire thermostat loop. A cleanup method is executed through a finally block to turn off LEDs, clear the LCD, close the serial connection, and release GPIO resources when the program ends.

Another important improvement is the use of a small hysteresis value around the set point. Instead of allowing sensor fluctuations immediately around the target temperature to repeatedly switch system behavior, the enhanced control logic uses a half-degree dead band. This demonstrates consideration of real-world system behavior and the trade-off between responsiveness and

stability. The result is a cleaner and more professional implementation while preserving the purpose and hardware behavior of the original artifact.

3. Course Outcome Coverage

The enhancement substantially supports the course outcome requiring the use of well-founded and innovative techniques, skills, and tools in computing practices to implement solutions that deliver value and accomplish industry-specific goals. The object-oriented redesign, centralized configuration, input validation, error handling, resource cleanup, and hysteresis logic demonstrate stronger software engineering practices than the original version.

The enhancement also supports the outcome involving the design and evaluation of computing solutions using computer science practices and standards while managing design trade-offs. For example, adding hysteresis requires balancing immediate reaction to a sensor reading against the need for stable control behavior. Encapsulation also introduces more structure than a small script requires, but that additional structure improves maintainability and prepares the project for future testing or expansion.

I also made progress toward the security-minded outcome. Although this thermostat is not a security-focused application, the enhancement applies defensive design principles by validating external sensor data, constraining user-controlled values, handling communication failures, and ensuring hardware resources are returned to a safe state. My outcome-coverage plan therefore remains centered on software design and engineering, with secondary evidence for algorithmic design and secure/defensive programming practices.

Finally, the narrative and the clearly labeled enhancement comments support the professional communication outcome by explaining technical decisions in language that can be understood by an instructor, reviewer, or future employer. The code comments are intended to make the improvements traceable rather than simply presenting a final product without evidence of the engineering decisions behind it.

4. Reflection on the Enhancement Process

Enhancing this artifact reinforced the difference between code that works and code that is engineered for continued use. In the original version, global variables made the program short and easy to follow at first, but they also meant that unrelated functions depended on shared state. Moving that state into a controller class made the relationships between mode, set point, hardware, and control behavior more explicit. I learned that encapsulation is valuable even in a relatively small embedded application because hardware projects can quickly become difficult to maintain as additional features are added.

One challenge was improving the project without overengineering it. I wanted the enhancement to clearly demonstrate growth, but I also wanted the final artifact to remain recognizable and manageable as a single Python file. I therefore avoided creating many modules and folders and instead used a single controller class, named constants, focused helper methods, and clearly separated responsibilities inside one file.

A second challenge was designing reliable behavior for a hardware-dependent application that cannot be fully exercised without the Raspberry Pi and connected components. To address this, I

kept the hardware interfaces consistent with the original project and focused the enhancement on code structure, validation, and failure handling. I also verified that the enhanced Python file compiles successfully at the syntax level. Actual sensor, GPIO, LCD, and UART behavior should still be tested on the physical Raspberry Pi before final portfolio publication.

Overall, this enhancement helped me see how software design decisions affect reliability and future change. I improved not only what the thermostat does, but also how the code communicates intent, responds to unexpected conditions, and manages the hardware resources that it controls. Those are the areas that most clearly demonstrate my growth in software design and engineering.